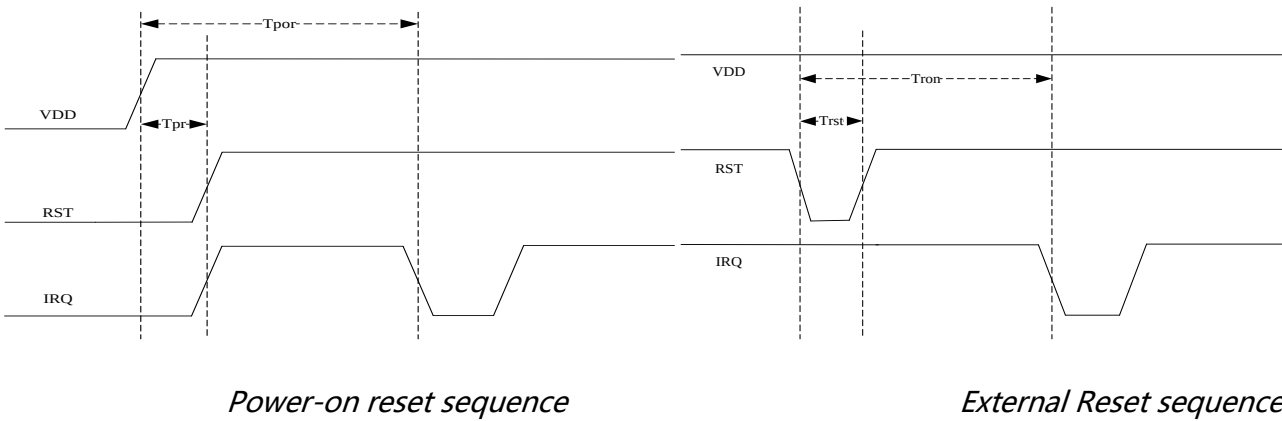


CST7xx 应用说明

上电时序说明

内置上电复位模块将使芯片保持在复位状态直至电压正常，当电压低于某阈值时，芯片也会被复位，但是如果断电不彻底可能会导致芯片复位失败，这时候需要通过复位脚复位整个芯片。电路设计时推荐将芯片复位脚连接到主控，不要悬空处理。

当外部复位引脚 RST 为低时将复位整个芯片



Parameter	Description	minimum	maximum	unit
T _{por}	Chip initialization time after power-on	100	-	mS
T _{pr}	RST pin delayed pull-up time	5	-	mS
T _{ron}	Chip reinitialization time after reset	100	-	mS
T _{rst}	reset pulse time	0.1	-	mS

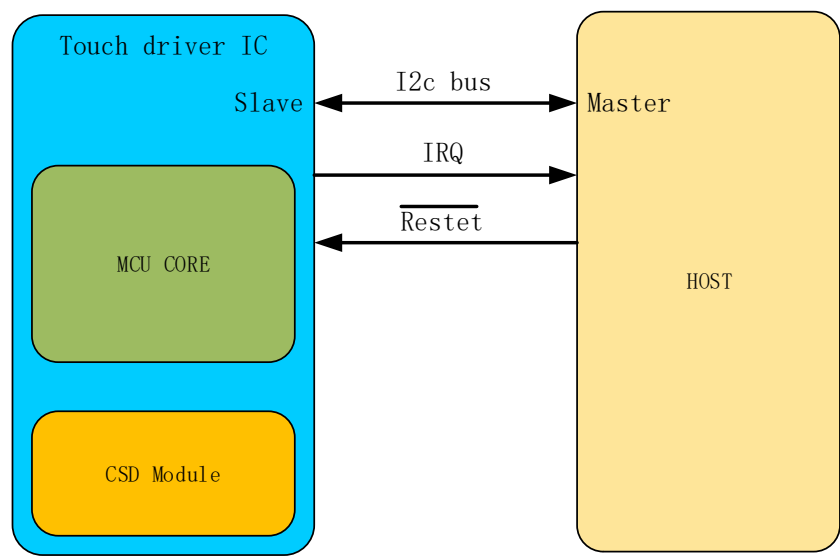
I²C 接口说明

主机和从机通信接口

如下图所示，主机和触摸 IC 之间存在以下三种通信

- 1、IIC 通信进行数据传输
- 2、有触摸数据上报通过中断通知主机，默认是下降沿触发

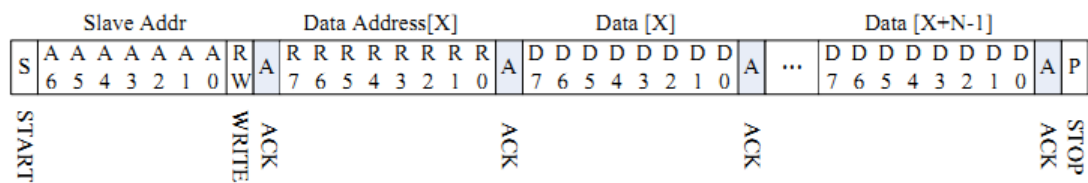
3、主机通过 Rst 脚复位触摸 IC，低电平有效



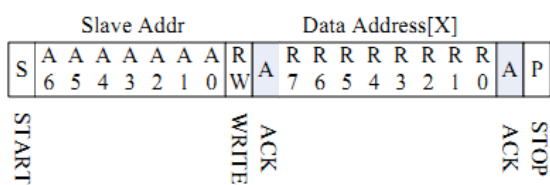
IIC 通信默认从设备地址是 **0x15 (7bit)** 带读写位就是 2A/2B，部分项目的设备地址可能不同，请咨询相应项目及工程人员。

I²C 读写模式

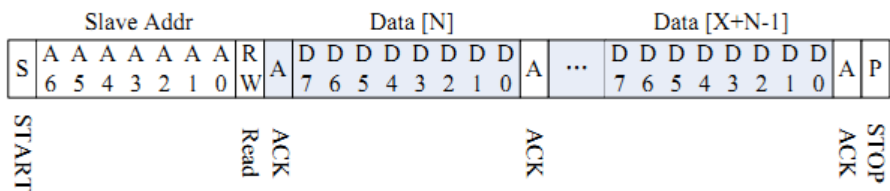
对从设备写多 Byte



写寄存器地址

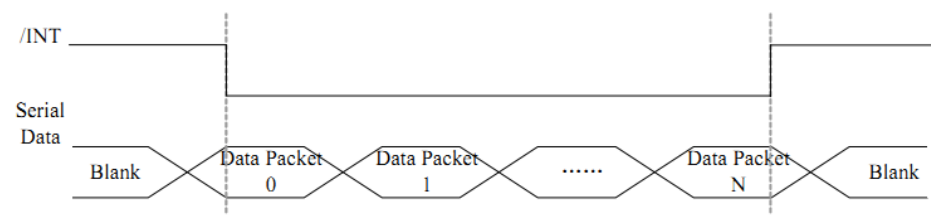


连续读多个 Byte

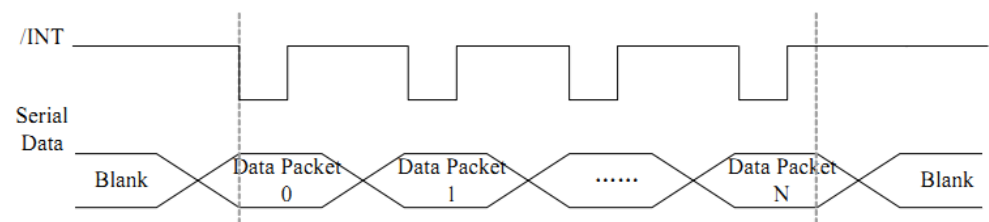


IRQ 信号响应分沿触发和电平触发两种模式

低电平触发方式



沿触发方式



寄存器说明

工作模式切换命令如下

工作模式	切换命令	描述
NOMAL	BE00	正常报点和手势上报
DBG_IDAC	F001	工厂测试数据获取
DBG_POS	BE01	工厂测试按键和坐标获取
DBG_RAW	BF06	原始值获取
DBG_SIG	BF07	differ 值获取

IC 上电后默认工作在 NOMAL 模式。

NOMAL 寄存器说明

Address	Name	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	说明	Access
00h	Reserve									预留	R/W
01h	Prox_state	Prox_state [7:0]								default: 00 far away: E0 near: C0	R
02h	touch num					touch points[3:0]					R

D0h	Ges_mode	Ges_mode [7:0]								Write 01H 进入 gesture 模式 Write 00H 退出 gesture 模式	W
...	...										
D3h	Gesture ID	Gesture[7:0]								手势模式使能才有效 double klick:0x24 up:0x22 down:0x23 left:0x20 right:0x21 C:0x34 e:0x33 m:0x32 O:0x30 S:0x46 V:0x54 W:0x31 Z:0x65	R

读取版本信息:

```

write_buf[0] = 0xA1;
hyn_wr_bytes (write_buf, 1, buf_read, 10);
ic->fw_project_id = (buf_read[0]<<24)|(buf_read[1]<<16)|(buf_read[2]<<8)|buf_read[3];
ic->fw_chip_type = buf_read[9];
ic->fw_ver = buf_read[5];
ic->fw_module_id = buf_read[7];

```

读点:

```

write_buf[0] = 0x02;
hyn_wr_bytes(write_buf,1,read_buf,(3+6*2));
rp_buf.rep_num = read_buf[0]&0x0F;
for(i = 0 ; i < 2 ; i++){
    id = (read_buf[3 + i*6] & 0xf0)>>4;
    event = (read_buf[1 + i*6]&0xC0)>>6; //press:0 hold:2 release:1 erro:3
    if(id > 1 || event==3) continue;
    rp_buf.pos_info[index].pos_id = id;
    rp_buf.pos_info[index].event = 1==event ? 0:1;
    rp_buf.pos_info[index].pos_x = ((u16)(read_buf[1 + i*6] & 0x0f)<<8) + read_buf[2 + i*6];
    rp_buf.pos_info[index].pos_y = ((u16)(read_buf[3 + i*6] & 0x0f)<<8) + read_buf[4 + i*6];
    index++;
}

```